

Modul A: Inteligentni sistemi za podporo odločanju

Domača naloga 3

Dominik Bernik¹

¹ Univerza v Ljubljani, Fakulteta za Elektrotehniko
E-pošta: db0975@student.uni-lj.si

1 Uvod

Pri tej domači nalogi smo morali narediti predikcijski model kvalitete za neznane izdelke. Imeli smo podani dve podatkovni bazi `studentData.mat` in `studentDataLarge.mat`. Podatkovni bazi vsebujeta N število izdelkov s 7 značilkami, ki jih moramo uporabiti za identifikacijo izdelka in kvaliteto specifičnega izdelka. Za identifikacijo neznanih izdelkov smo si lahko pomagali z algoritmom Gustafson–Kessel (GK), medtem ko smo za izračun kvalitete izdelka uporabili PSA + LS. Za analizo smo podatkovno bazo morali razdeliti na dva načina. Prvi način je bil podatkovno bazo razdeliti na 80/20 učno in testno množico brez sortiranja, drugi način pa s pomočjo sortiranja.

Kot podnaloge se je naloga razdelila na tri dele:

- **Model A:** identifikacija sistema s pomočjo algoritma GK na parametrih (2,6) ter implementacija PSA + LS za izračun kvalitete izdelka. Tu smo lahko uporabili vse značilke, ne samo 2 in 6, in naredili en model na cluster.
- **Model B:** izračun kvalitet brez uporabe manjših clusterjev.
- **Model C:** identifikacija na vseh značilkah, ne samo na (2,6).

2 Metodologija

Pri tej domači nalogi smo se osredotočili na tri večje algoritme: algoritem za mehko razvrščanje v gruče Gustafson–Kessel (GK) (ang. *Fuzzy C-Means*), analizo glavnih komponent (ang. *Principal Component Analysis*, PCA) in metodo najmanjših kvadratov (ang. *Least Squares*, LS).

Za analizo smo imeli na voljo dve podatkovni bazi, `studentData.mat` in `studentDataLarge.mat`. Prva vsebuje 687×7 vhodnih in 687×1 izhodnih podatkov, medtem ko večja podatkovna baza vsebuje 2060×7 vhodnih in 2060×1 izhodnih podatkov. Glavna pomanjkljivost obeh podatkovnih baz je, da vzorci ne vsebujejo informacij o pripadnosti razredom. Zato je bil prvi korak normalizacija vseh podatkov, ki smo jih nato s pomočjo algoritma GK klasificirali po metodi nenadzorovanega učenja. Z novo pridobljenimi gručami (ang. *clustering*) smo lahko nadaljevali z reševanjem PCA + LS za vsako specifično gručo.

Postopek klasifikacije je bil najprej izveden na dveh normaliziranih vhodnih podatkih (2 in 6) in nato ponovno na vseh, tako kot je predvideno za Model A in Model C. Pri postopku za Model B pa smo morali spremeniti metodologijo računanja končne vrednosti, saj je bilo treba ustvariti le en model za vse podatke, medtem ko je bilo pri Modelu A in Modelu C treba zgraditi lasten model za vsako gručo.

2.1 Normalizacija in standardizacija podatkovne baze

Pri pripravi podatkovne baze smo se morali najprej odločiti, ali bomo podatke normalizirali ali standardizirali.

Pri postopku normalizacije (angl. *Min-Max scaling*) se vse vrednosti preslikajo na interval med 0 in 1. To je izjemno uporabno pri algoritmu GK (Gustafson–Kessel), saj je ta algoritem, podobno kot večina metod mehke gručne analize, občutljiv na različne skale (merske enote) vhodnih spremenljivk. Če podatkov ne bi normalizirali, bi spremenljivke z večjimi absolutnimi vrednostmi prevladale pri izračunu razdalj (kovariančnih matrik), kar bi popačilo obliko in položaj gruč. Normalizacija tako zagotovi, da imajo vsi atributi enak vpliv na postopek razvrščanja.

Postopek standardizacije (angl. *Z-score normalization*) pa vse vrednosti premakne tako, da je aritmetična sredina podatkov enaka 0, standardni odklon pa 1. S tem podatke centriramo okoli izhodišča in odpravimo vpliv variance posameznih spremenljivk, pri čemer pa podatki nimajo strogo omejenih meja (kot sta 0 in 1) in lahko vsebujejo tudi negativne vrednosti.

Za postopek normalizacije sta bili uporabljeni naslednji enačbi:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (1)$$

$$y_{norm} = \frac{y - y_{min}}{y_{max} - y_{min}} \quad (2)$$

Za postopek standardizacije pa se uporabljata enačbi:

$$X_{std} = \frac{X - \mu_X}{\sigma_X} \quad (3)$$

$$y_{std} = \frac{y - \mu_y}{\sigma_y} \quad (4)$$

kjer μ predstavlja aritmetično sredino (povprečje), σ pa standardni odklon posamezne spremenljivke.

Tekom preizkusov smo ugotovili, da je normalizacija prinesla boljše rezultate (stabilnejše konvergenco algoritma GK in natančnejše oblikovanje gruči), zato je bila uporabljena tudi v končni iteraciji modela.

2.2 Implementacija algoritma FCM za mehko nenadzorovano učenje

Glavna razlika med klasičnim trdim razvrščanjem v gruče (angl. *Hard K-Means*) in mehkih C-sredin (angl. *Fuzzy C-Means*, FCM) je v tem, da ima tu vsak podatek stopnjo pripadnosti posamezni gruči, ki jo opisuje pripadnostna vrednost U_{ik} . Ta vrednost ne predstavlja klasične statistične verjetnosti, temveč stopnjo pripadnosti vzorca določeni gruči. Za te vrednosti velja naslednji omejitveni pogoj:

$$\sum_{i=1}^c U_{ik} = 1, \quad \forall k \quad (5)$$

V zgornji enačbi c predstavlja število razredov, k pa indeks posameznega vzorca.

V postopku učenja se skozi iteracije izmenično posodablja pripadnosti podatkov in položaji središč posameznih gruči. Najprej se za vsak vzorec izračuna nova matrika pripadnosti U_{ik} na podlagi trenutnih središč gruči z naslednjo enačbo:

$$U_{ik} = \frac{\|\mathbf{x}_k - \mathbf{v}_i\|^{-2/(m-1)}}{\sum_{j=1}^c \|\mathbf{x}_k - \mathbf{v}_j\|^{-2/(m-1)}} \quad (6)$$

kjer $\mathbf{x}_k$ predstavlja k -ti vhodni vzorec, $\mathbf{v}_i$ središče i -te gruče, parameter m ($m > 1$) fuzzifier, ki določa stopnjo mehkobe razvrščanja, $\|\cdot\|$ pa predstavlja evklidsko razdaljo.

Takoj za tem se na podlagi novih pripadnosti prilagodijo še središča posameznih gruči $\mathbf{v}_i$, kar se izvede s pomočjo enačbe:

$$\mathbf{v}_i = \frac{\sum_k U_{ik}^m \mathbf{x}_k}{\sum_k U_{ik}^m} \quad (7)$$

Ta dva koraka se izmenično ponavljata, dokler algoritem ne konvergira oziroma dokler se razlika med položaji središč v dveh zaporednih iteracijah ne spusti pod vnaprej določeno toleranco.

2.3 Evalvacija delovanja

Za preverjanje delovanja algoritma GK, smo poskusili minimizirati naslednjo enačbo:

$$J(U, V) = \sum_{i=1}^c \sum_{k=1}^N U_{ik}^m \|\mathbf{x}_k - \mathbf{v}_i\|_2^2 \quad (8)$$

Za evalvacijo smo prav tako uporabili naslenje pristope:

- **Particijski koeficient (PC):** Pove nam, kako "ostra" oziroma izrazita je dobljena razdelitev podatkov in se izračunamo po naslednji enačbi:

$$PC = \frac{1}{N} \sum_{k=1}^N \sum_{i=1}^c U_{ik}^2 \quad (9)$$

kjer višja vrednost koeficienta pomenijo čistejšo razdelitev z manj prekrivanja med posameznimi grupami.

- **Xie-Beni indeks (XB):** Ocenjuje razmerje med kompaktnostjo gruči in njihovo medsebojno ločenostjo. Definiran je z enačbo:

$$XB = \frac{\sum_{i=1}^c \sum_{k=1}^N U_{ik}^m \|\mathbf{x}_k - \mathbf{v}_i\|^2}{N \cdot \min_{i \neq j} \|\mathbf{v}_i - \mathbf{v}_j\|^2} \quad (10)$$

Števec v enačbi meri skupno kompaktnost gruči, medtem ko imenovalec predstavlja minimalno razdaljo med središči dveh različnih gruči. Pri tem pravilu velja, da nižja kot je vrednost, bolj uspešno je razvrščanje.

- **Povprečni koeficient silhuete (SIL):** Se računa na podlagi trde dodelitve vzorcev (angl. *hard assignment*), kjer posameznemu vzorcu k pripišemo indeks gruče z najvišjo pripadnostjo: $\hat{c}_k = \arg \max_i U_{ik}$. Za vsak vzorec se najprej izračuna vrednost silhuete:

$$s_k = \frac{b_k - a_k}{\max(a_k, b_k)} \quad (11)$$

kjer je a_k povprečna razdalja vzorca k do vseh ostalih vzorcev znotraj njegove lastne gruče, b_k pa povprečna razdalja do vzorcev v najbližji sosednji gruči [cite: 61]. Celotni indeks SIL predstavlja povprečje čez vse vzorce:

$$SIL = \frac{1}{N} \sum_{k=1}^N s_k \quad (12)$$

Vrednosti indeksa ležijo na intervalu $SIL \in [-1, 1]$, pri čemer višje vrednosti nakazujejo pravilnejšo in stabilnejšo strukturo gruči.

2.4 Lokalni napovedni modeli z analizo glavnih komponent (PCA) in metodo najmanjših kvadratov (LS)

Po pravilnem razvrščanju produktov smo za vsak definiran razred izvedli analizo glavnih komponent (ang. Principal Component Analysis, PCA). Namen metode PCA je zmanjšanje dimenzionalnosti podatkov ter izločitev manj pomembnih značilk ob hkratnem ohranjanju čim več informacij, ki jih vsebujejo vhodni podatki.

Prvi korak pri izračunu PCA je določitev povprečne vrednosti posameznih vhodnih parametrov:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad (13)$$

Na podlagi izračunanega povprečja podatke centriramo:

$$X_c = X - \mathbf{1}\mu^T \quad (14)$$

kjer je $\mathbf{1}$ vektor enic ustrezne dimenzije. Centrirani podatki imajo povprečno vrednost enako nič, kar omogoča pravilno analizo variance posameznih značilk.

V naslednjem koraku izračunamo kovariančno matriko:

$$\Sigma = \frac{1}{N-1} X_c^T X_c \quad (15)$$

Nad kovariančno matriko nato izvedemo lastni razcep (ang. eigendecomposition), pri katerem določimo lastne vrednosti λ_i in pripadajoče lastne vektorje. Lastne vrednosti predstavljajo količino variance, ki jo pojasni posamezna glavna komponenta, zato jih uredimo po velikosti v padajočem vrstnem redu. Komponente z največjimi lastnimi vrednostmi vsebujejo največ informacij o podatkih, medtem ko komponente z majhnimi lastnimi vrednostmi prispevajo zanemarljiv delež variance in jih lahko odstranimo.

Za določitev števila ohranjenih komponent uporabimo kumulativni delež pojasnjene variance (ang. Explained Variance Ratio, EVR):

$$EVR(k) = \frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^p \lambda_j} \quad (16)$$

kjer je k število izbranih glavnih komponent, p pa skupno število komponent. Izberemo najmanjše število komponent k , pri katerem EVR doseže vnaprej določen prag (npr. 95 %). Na ta način odstranimo manj pomembne komponente in zmanjšamo dimenzionalnost podatkov ob minimalni izgubi informacij.

Za določitev parametrov linearnega modela smo uporabili metodo najmanjših kvadratov (LS), katere cilj je minimizirati vsoto kvadratov napak med dejanskimi in napovedanimi vrednostmi. Parametre modela θ določimo z rešitvijo normalnih enačb:

$$\theta = (X^T X)^{-1} X^T y \quad (17)$$

kjer matrika X predstavlja vhodne podatke po predprocesiranju, vektor y pa pripadajoče normalizirane izhodne vrednosti. Izračunani parametri θ nato določajo vpliv posameznih vhodnih spremenljivk na napovedano kakovost izdelka.

Po določitvi parametrov modela θ lahko izvedemo napoved kakovosti izdelka. Vhodno matriko najprej razširimo s stolpcem enic, s čimer omogočimo upoštevanje prostega člena modela:

$$X_e = [1, X] \quad (18)$$

Napovedane vrednosti nato izračunamo z linearnim modelom:

$$\hat{y} = X_e \theta \quad (19)$$

kjer $\hat{y}$ predstavlja vektor napovedanih vrednosti kakovosti izdelkov, θ pa vektor parametrov, določenih z metodo najmanjših kvadratov. Model tako za vsak vzorec izračuna ocenjeno kakovost na podlagi vhodnih značilk oziroma izbranih glavnih komponent.

2.5 Evalvacija modela LS

Za evalvacijo napovednega modela smo uporabili metriki RMSE (ang. Root Mean Square Error) in $RMSE_{mean}$.

RMSE smo uporabili za oceno splošne uspešnosti modela, medtem ko nam je $RMSE_{mean}$ omogočil podrobnejši vpogled v delovanje modela znotraj posameznih razredov (clusterjev).

RMSE meri povprečno odstopanje napovedanih vrednosti od dejanskih vrednosti in ga izračunamo po enačbi:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2} \quad (20)$$

kjer je N število vzorcev, $\hat{y}_i$ napovedana vrednost modela, y_i pa dejanska vrednost.

Za analizo posameznega razreda smo izračunali povprečno vrednost izhoda znotraj razreda k :

$$\bar{y}_k = \frac{1}{N_k} \sum_{i=1}^{N_k} y_i \quad (21)$$

kjer je N_k število vzorcev v razredu k .

Nato smo izračunali metriko $RMSE_{mean}$, ki meri odstopanje napovedi od povprečne vrednosti posameznega razreda:

$$RMSE_{mean,k} = \sqrt{\frac{1}{N_k} \sum_{i=1}^{N_k} (\hat{y}_{i,k} - \bar{y}_k)^2} \quad (22)$$

kjer indeks k označuje obravnavani razred. S primerjavo vrednosti RMSE in $RMSE_{mean}$ lahko ocenimo, ali model znotraj posameznega razreda zajame več informacij kot zgolj uporaba povprečne vrednosti izhoda tega razreda.

3 Implementacija napovedovanja (angl. forecasting) in rezultati

Implementacija se v našem primeru razdeli na 4 večje dele. Prva dva dela se osredotočata na manjšo podatkovno bazo `studentData.mat`, kjer jo enkrat razdelimo na 80/20 split brez resortiranja in nato 80/20 split s sortiranjem. Nato to ponovimo še za `studentDataLarge.mat`. Vsako delo prav tako vsebuje Modul A, Modul B in Modul C.

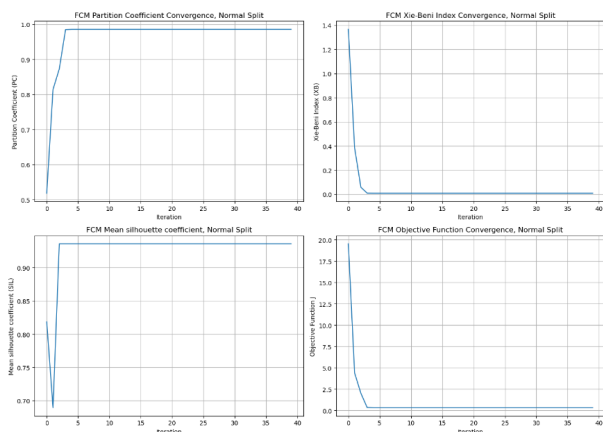
3.1 `studentData.mat`, 80/20 enakomerno porazdeljeno

3.1.1 Model A

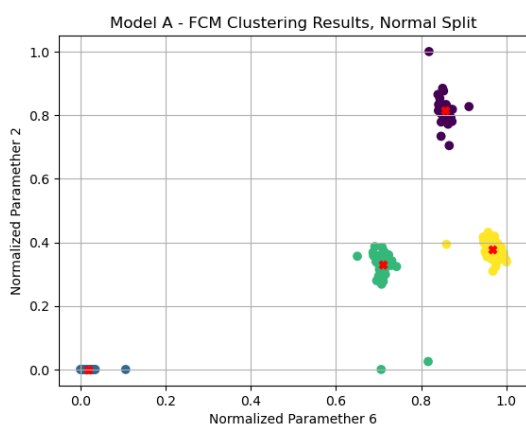
Po pravilnem sortiranju podatkov smo se najprej osredotočili na učenje FCM s pomočjo algoritma AG. To nam je omogočilo, da s pomočjo nenadzorovanega učenja identificiramo klastre oziroma razrede. Za evalvacijo delovanja smo uporabili indekse PC , XB , SIL in J_{hist} .

Tabela 1: Evalvacijske metrike za FCM

Metrika	Vrednost
Partition Coefficient (PC)	0.985619
Xie-Beni Index (XB)	0.008905
Mean Silhouette Coefficient (SLI)	0.935777



Slika 1: Evalvacijske metrike za FCM



Slika 2: Razdelitev vzorcev po oznakah

Kot je vidno iz slike 1 in 2, centri središč vsakega razreda lepo konvergirajo proti njihovi končni vrednosti. Po pridobitvi oznak smo za vsak razred naredili PSA, kjer smo izločili nepotrebne vhodne spremenljivke.

Tabela 2: PCA komponente po razredih (učenje)

Razred	PCA komponente (ohranjenih)
0	4
1	3
2	3
3	3

Po uspešni implemetaciji PSA smo s pomočjo metode LS naredili predikcijski model, kjer smo prili do naslednjih rezultatov:

Tabela 3: RMSE po razredih (učenje in testiranje)

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.048581	0.048371
1	0.003841	0.019693
2	0.021801	0.022302
3	0.035453	0.079249

Iz tabele 3 je razvidno, da je $RMSE_{mean}$ pri učenju najnižji za razred 1 (0.003841), medtem ko je $RMSE_{mean}$ pri testiranju najnižja za razred 1 (0.019693).

3.1.2 Model B

Pri Modelu B smo naredili globalni model brez clusteringa, kjer uporabimo en sam PCA za vse podatke in nato en LS model za predikcijo kvalitete. Prav tako smo naredili dve iteraciji, prva s PCA + LS in druga brez PCA.

Tabela 4: Globalni PCA komponente

Model	PCA komponente (ohranjenih)
PCA + LS	4
Samo LS	/

Pri globalnem modelu s PCA smo obdržali 4 glavne komponente, ki pojasnijo 95.0% variance. Po implementaciji PCA smo s pomočjo metode LS naredili predikcijski model, kjer smo prišli do naslednjih rezultatov:

Tabela 5: RMSE za globalni model PCA + LS (učenje in testiranje)

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.088292	0.100837
1	0.008912	0.019091
2	0.060721	0.041081
3	0.053398	0.094035

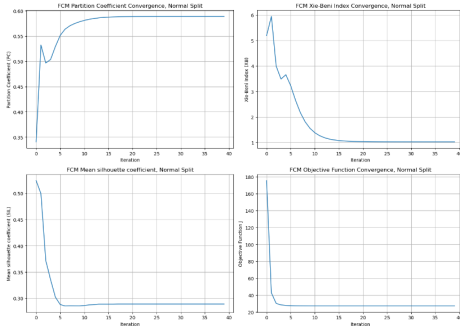
Tabela 6: RMSE za globalni model samo LS (učenje in testiranje)

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.082351	0.084472
1	0.006584	0.019992
2	0.031883	0.035527
3	0.031981	0.055206

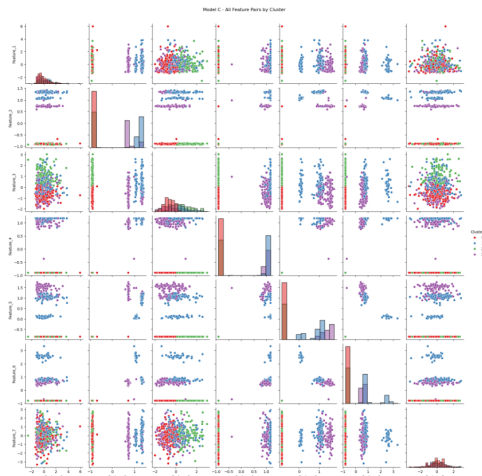
Iz tabele 5 in 6 je razvidno, da model brez PCA dosega boljše rezultate in da pri učenju RMSE doseže 0.0968, LS, v primerjavi z 0.1059, ko pa imamo PCA + LS. Pri testni množici pa dosežemo 0.1105, pri PSA + LS pa dosežemo RMSE vrednost 0.1213. Prav tako so RMSE_mean vrednosti po razredih nižje pri modelu brez PCA, še posebej pri razredu 0 in 3 na učni množici.

3.1.3 Model C

Pri Modelu C smo ponovili postopek iz Modela A, vendar tokrat z uporabo vseh značilk, ne samo 2 in 6. Tu smo prišli do naslednjih rezultatov:



Slika 3: Evalvacija metrik FCM na vseh značilkah



Slika 4: Porazdelitev vzorcev po oznakah

Kot je razvidno iz slik 4 in 3, so evalvacije tokrat slabše, saj SIL ne konvergira proti najvišji vrednosti. Prav tako vrednosti ne konvergirajo proti najvišji/najnižji vrednosti.

Za slabšo identifikacijo tudi pride zaradi upoštevanja značilk kot so 1, 3 in 6, ki imajo veliko pokrivanja med razredi, kar lahko prinese nesigurnost v odločevanje.

Kljub temu smo tu poskusili delovanje PCA + LS predikcije in prišli do naslednjih rezultatov:

Tabela 7: PCA komponente po razredih (Model C)

Razred	PCA komponente (ohranjenih)
0	4
1	4
2	3
3	5

Po uspešni implementaciji PCA smo s pomočjo metode LS naredili predikcijski model, kjer smo prišli do naslednjih rezultatov:

Tabela 8: RMSE po razredih (učenje in testiranje) - Model C

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.060469	0.040136
1	0.026230	0.028928
2	0.013264	0.014351
3	0.085118	0.111069

Iz tabele 8 je razvidno, da je RMSE_mean pri učenju najnižji za razred 2 (0.013264), medtem ko je RMSE pri testiranju najvišji za razred 3 (0.111069). Prav tako opazimo, da ima razred 3 največje število ohranjenih PCA komponent (5), kar nakazuje na večjo kompleksnost podatkov v tem razredu.

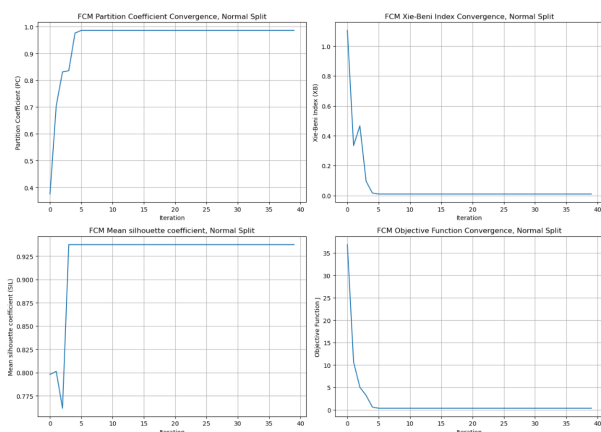
3.2 studentData.mat, 80/20 brez sortiranja

3.2.1 Model A

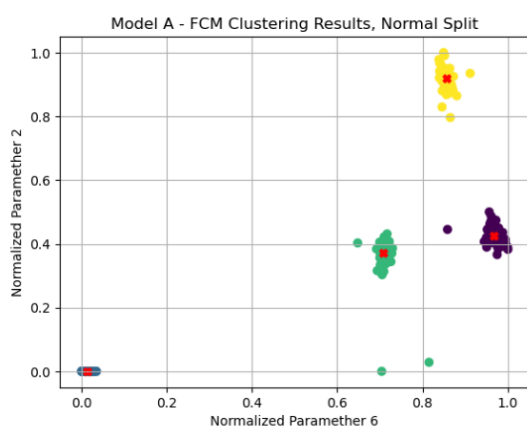
Pri tem delu smo vzeli prvih 80% vzorcev kot učno množico in preostalih 20% kot testno množico. Podatke smo najprej uporabili za učenje FCM s pomočjo algoritma AG. To nam je omogočilo, da s pomočjo nenadzorovanega učenja identificiramo klastre oziroma razrede. Za evalvacijo delovanja smo tu ponovno uporabili PC , XB , SIL in J_{hist} .

Tabela 9: Evalvacijske metrike za FCM

Metrika	Vrednost
Partition Coefficient (PC)	0.985933
Xie-Beni Index (XB)	0.009099
Mean Silhouette Coefficient (SLI)	0.93755



Slika 5: Evalvacijske metrike za FCM



Slika 6: Razdelitev vzorcev po oznakah

Kot je vidno iz slike 5 in 6, centri središč vsakega razreda lepo konvergirajo proti njihovi končni vrednosti. Po pridobitvi oznak smo za vsak razred naredili PSA, kjer smo izločili nepotrebne vhodne spremenljivke.

Tabela 10: PCA komponente po razredih (učenje)

Razred	PCA komponente (ohranjenih)
0	3
1	3
2	4
3	3

Po uspešni implemetaciji PSA smo s pomočjo metode LS naredili predikcijski model, kjer smo prili do naslednjih rezultatov:

Tabela 11: RMSE po razredih (učenje in testiranje)

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.019101	0.019935
1	0.008155	0.024691
2	0.053434	0.047627
3	0.017796	0.117409

Iz tabele 11 je razvidno, da je $RMSE_{mean}$ pri učenju najnižji za razred 1 (0.008155), medtem ko je $RMSE_{mean}$ pri testiranju najnižja za razred 0 (0.019935). Najvišja $RMSE_{mean}$ pri testiranju je za razred 3 (0.117409), vendar je treba upoštevati, da ima razred 3 le 2 testna primera.

3.2.2 Model B

Pri Modelu B smo naredili globalni model brez clusteringa, kjer uporabimo en sam PCA za vse podatke in nato en LS model za predikcijo kvalitete. Prav tako smo naredili dve iteraciji, prva s PCA + LS in druga brez PCA.

Tabela 12: Globalni PCA komponente

Model	PCA komponente (ohranjenih)
PCA + LS	4
Samo LS	/

Pri globalnem modelu s PCA smo obdržali 4 glavne komponente, ki pojasnijo 95.0% variance. Po implementaciji PCA smo s pomočjo metode LS naredili predikcijski model, kjer smo prišli do naslednjih rezultatov:

Tabela 13: RMSE za globalni model PCA + LS (učenje in testiranje)

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.061705	0.083095
1	0.006620	0.017870
2	0.088970	0.106944
3	0.032504	0.140286

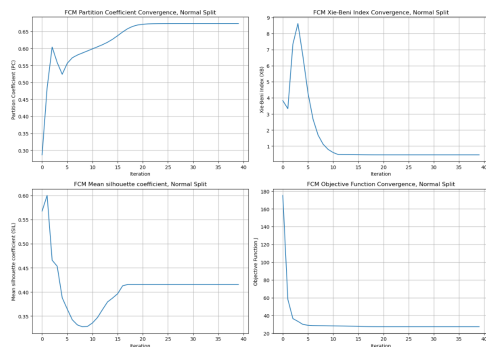
Tabela 14: RMSE za globalni model samo LS (učenje in testiranje)

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.030373	0.035267
1	0.002250	0.021153
2	0.086210	0.089895
3	0.026049	0.002033

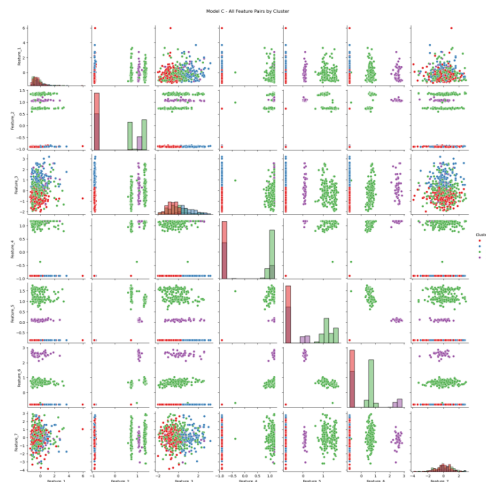
Iz tabele 13 in 14 je razvidno, da model brez PCA dosega boljše rezultate. Pri učenju RMSE doseže 0.1039 (samo LS) v primerjavi z 0.1105 (PCA + LS). Pri testni množici pa dosežemo 0.1155 (samo LS), pri PCA + LS pa dosežemo RMSE vrednost 0.1339. Prav tako so $RMSE_{mean}$ vrednosti po razredih nižje pri modelu brez PCA, še posebej pri razredu 0 in 3. Pri razredu 3 na testni množici model brez PCA doseže izjemno nizko vrednost 0.002033, vendar je treba upoštevati, da ima razred 3 le 2 testna primera.

3.2.3 Model C

Pri Modelu C smo ponovili postopek iz Modela A, vendar tokrat z uporabo vseh značilk, ne samo 2 in 6. Tu smo prišli do naslednjih rezultatov:



Slika 7: Evalvacija metrik FCM na vseh značilkah



Slika 8: Porazdelitev vzorcev po oznakah

Kot je razvidno iz slik 7 in 8, so evalvacije tokrat slabše, saj SIL ne konvergira proti najvišji vrednosti. Prav

tako vrednosti ne konvergirajo proti najvišji/najnižji vrednosti.

Do slabše identifikacije pride tudi zaradi upoštevanja značilk, kot so 1, 3 in 6, ki imajo veliko pokrivanja med razredi, kar lahko prinese negotovost v odločanje.

Tabela 15: PCA komponente po razredih (Model C)

Razred	PCA komponente (ohranjenih)
0	4
1	3
2	5
3	3

Po uspešni implementaciji PCA smo s pomočjo metode LS naredili predikcijski model, kjer smo prišli do naslednjih rezultatov:

Tabela 16: RMSE po razredih (učenje in testiranje) - Model C

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.058547	0.037559
1	0.013422	0.030188
2	0.068738	0.051204
3	0.017796	0.117409

Iz tabele 16 je razvidno, da je $RMSE_{mean}$ pri učenju najnižji za razred 1 (0.013422), medtem ko je $RMSE_{mean}$ pri testiranju najvišji za razred 3 (0.117409). Prav tako opazimo, da ima razred 2 največje število ohranjenih PCA komponent (5), kar nakazuje na večjo kompleksnost podatkov v tem razredu. Pri razredu 3 na testni množici je vrednost visoka, vendar je treba upoštevati, da ima razred 3 le 2 testna primera.

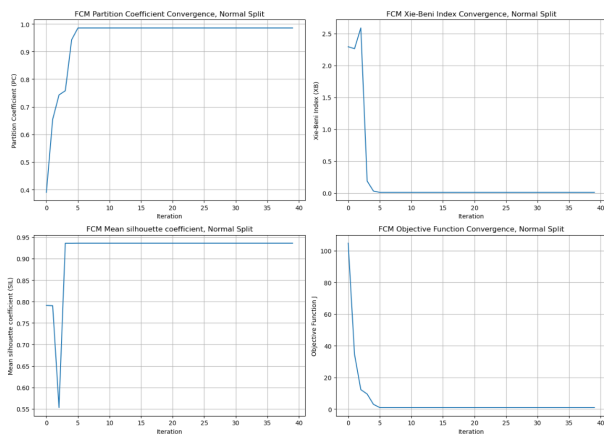
3.3 studentDataLarge.mat, 80/20 enakomerno porazdeljeno

3.3.1 Model A

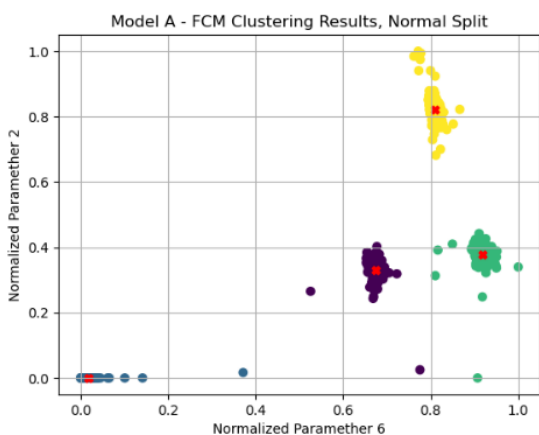
Pri tem delu naloge smo se osredotočili na večjo podatkovno bazo. Tu smo jo razdelili na 80/20 in jo enakomerno porazdelili (angl. shuffle). Nato smo s pomočjo nenadzorovanega učenja identificirali klastre oziroma razrede in za evalvacijo uporabili PC , XB , SIL in J_{hist} .

Tabela 17: Evalvacijske metrike za FCM

Metrika	Vrednost
Partition Coefficient (PC)	0.98553
Xie-Beni Index (XB)	0.009215
Mean Silhouette Coefficient (SLI)	0.93595



Slika 9: Evalvacijske metrike za FCM na veliki podatkovni bazi



Slika 10: Razdelitev vzorcev po oznakah

Kot je vidno iz slik 9 in 10, centri središč vsakega razreda lepo konvergirajo proti njihovi končni vrednosti. Po pridobitvi oznak smo za vsak razred naredili PCA, kjer smo izločili nepotrebne vhodne spremenljivke.

Tabela 18: PCA komponente po razredih (Model A - Large)

Razred	PCA komponente (ohranjenih)
0	4
1	3
2	4
3	4

Po uspešni implementaciji PCA smo s pomočjo metode LS naredili predikcijski model, kjer smo prišli do naslednjih rezultatov:

Tabela 19: RMSE po razredih (učenje in testiranje) - Large

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.063450	0.125002
1	0.006907	0.009777
2	0.056720	0.029074
3	0.058431	0.052581

Iz tabele 19 je razvidno, da je $RMSE_{mean}$ pri učenju najnižji za razred 1 (0.006907), medtem ko je $RMSE_{mean}$ pri testiranju najnižja prav tako za razred 1 (0.009777). Najvišja napaka pri testiranju je pri razredu 0 (0.125002).

3.3.2 Model B

Pri Modelu B smo naredili globalni model brez clusteringa, kjer uporabimo en sam PCA za vse podatke in nato en LS model za predikcijo kvalitete. Prav tako smo naredili dve iteraciji, prva s PCA + LS in druga brez PCA.

Tabela 20: Globalni PCA komponente (Large)

Model	PCA komponente (ohranjenih)
PCA + LS	3
Samo LS	/

Pri globalnem modelu s PCA smo obdržali 3 glavne komponente, ki pojasnijo 95.0% variance. Po implementaciji PCA smo s pomočjo metode LS naredili predikcijski model, kjer smo prišli do naslednjih rezultatov:

Tabela 21: RMSE za globalni model PCA + LS (učenje in testiranje) - Large

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.069989	0.112033
1	0.006765	0.023705
2	0.064178	0.056238
3	0.046770	0.060390

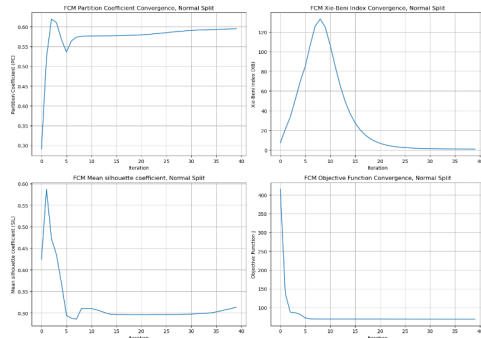
Iz tabele 21 in 22 je razvidno, da model brez PCA dosega boljše rezultate. Pri učenju RMSE doseže 0.1076 (samo LS) v primerjavi z 0.1176 (PCA + LS). Pri testni množici pa dosežemo 0.0892 (samo LS), pri PCA + LS pa dosežemo RMSE vrednost 0.0978. Prav tako so $RMSE_{mean}$ vrednosti po razredih nižje pri modelu brez PCA, še posebej pri razredu 0, 2 in 3 na testni množici.

3.3.3 Model C

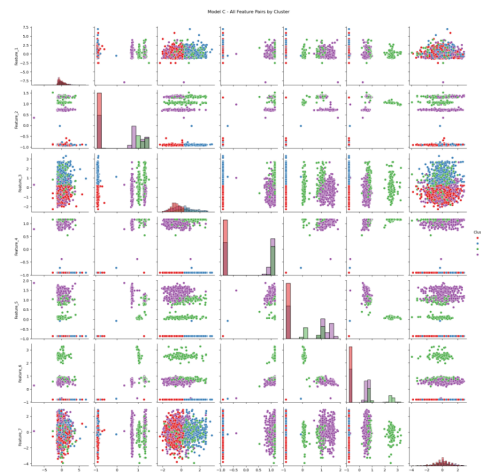
Pri Modelu C smo ponovili postopek iz Modela A, vendar tokrat z uporabo vseh značilk, ne samo 2 in 6. Tu smo prišli do naslednjih rezultatov:

Tabela 22: RMSE za globalni model samo LS (učenje in testiranje) - Large

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.057028	0.103228
1	0.008680	0.013734
2	0.047878	0.031795
3	0.038189	0.028247



Slika 11: Evalvacija metrik FCM na vseh značilkah (Large)



Slika 12: Porazdelitev vzorcev po oznakah (Large)

Kot je razvidno iz slik 12 in 11, so evalvacije tokrat slabše, saj SIL ne konvergira proti najvišji vrednosti. Prav tako vrednosti ne konvergirajo proti najvišji/najnižji vrednosti.

Do slabše identifikacije pride tudi zaradi upoštevanja značilk, kot so 1, 3 in 6, ki imajo veliko pokrivanja med razredi, kar lahko prinese negotovost v odločanje.

Kljub temu smo tu poskusili delovanje PCA + LS predikcije in prišli do naslednjih rezultatov:

Tabela 23: PCA komponente po razredih (Model C - Large)

Razred	PCA komponente (ohranjenih)
0	3
1	3
2	3
3	5

Po uspešni implementaciji PCA smo s pomočjo metode LS naredili predikcijski model, kjer smo prišli do naslednjih rezultatov:

Tabela 24: RMSE po razredih (učenje in testiranje) - Model C (Large)

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.004269	0.004282
1	0.010431	0.018053
2	0.044218	0.041797
3	0.054961	0.048925

Iz tabele 24 je razvidno, da je $RMSE_{mean}$ pri učenju najnižji za razred 0 (0.004269), medtem ko je $RMSE_{mean}$ pri testiranju prav tako najnižji za razred 0 (0.004282). Najvišjo napako pri testiranju ima razred 3 (0.048925). Prav tako opazimo, da ima razred 3 največje število ohranjenih PCA komponent (5), kar nakazuje na večjo kompleksnost podatkov v tem razredu.

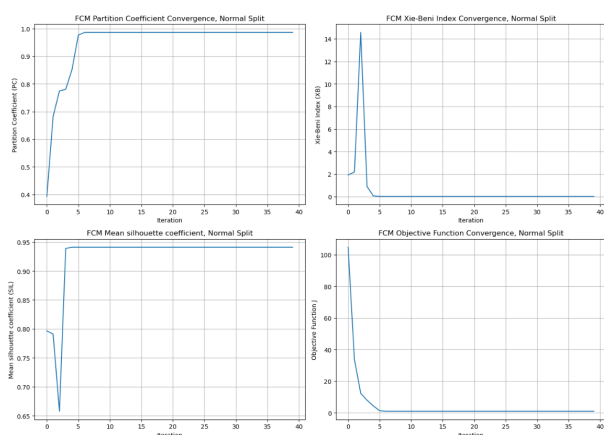
3.4 studentDataLarge.mat, 80/20 brez sortiranja

3.4.1 Model A

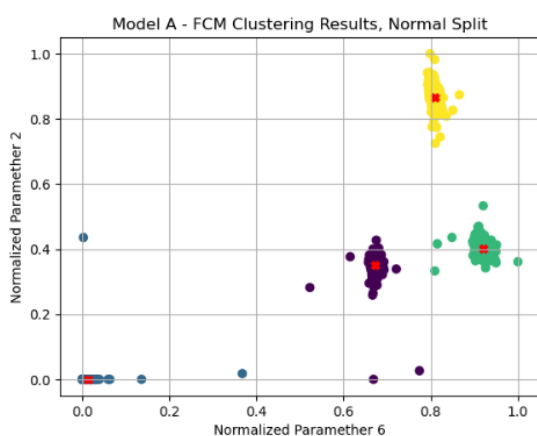
Pri tem delu naloge smo se osredotočili na večjo podatkovno bazo. Tu smo jo razdelili na 80/20 brez mešanja (angl. without shuffle). Nato smo s pomočjo nenadzorovanega učenja identificirali klastre oziroma razrede in za evalvacijo uporabili PC , XB , SIL in J_{hist} .

Tabela 25: Evalvacijske metrike za FCM (brez sortiranja)

Metrika	Vrednost
Partition Coefficient (PC)	0.98656
Xie-Beni Index (XB)	0.00878
Mean Silhouette Coefficient (SLI)	0.94098



Slika 13: Evalvacijske metrike za FCM na veliki podatkovni bazi (brez sortiranja)



Slika 14: Razdelitev vzorcev po oznakah (brez sortiranja)

Kot je vidno iz slik 13 in 14, centri središč vsakega razreda lepo konvergirajo proti njihovi končni vrednosti. Po pridobitvi oznak smo za vsak razred naredili PCA, kjer smo izločili nepotrebne vhodne spremenljivke.

Tabela 26: PCA komponente po razredih (Model A - Large, brez sortiranja)

Razred	PCA komponente (ohranjenih)
0	4
1	3
2	4
3	3

Po uspešni implementaciji PCA smo s pomočjo metode LS naredili predikcijski model, kjer smo prišli do naslednjih rezultatov:

Tabela 27: RMSE po razredih (učenje in testiranje) - Large (brez sortiranja)

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.050827	0.048974
1	0.004466	0.010031
2	0.034174	0.070668
3	0.014890	0.134743

Iz tabele 27 je razvidno, da je $RMSE_{mean}$ pri učenju najnižji za razred 1 (0.004466), medtem ko je $RMSE_{mean}$ pri testiranju najnižja prav tako za razred 1 (0.010031). Najvišja napaka pri testiranju je pri razredu 3 (0.134743), vendar je treba upoštevati, da ima razred 3 le 6 testnih primerov.

3.4.2 Model B

Pri Modelu B smo naredili globalni model brez clusteringa, kjer uporabimo en sam PCA za vse podatke in nato en LS model za predikcijo kvalitete. Prav tako smo naredili dve iteraciji, prva s PCA + LS in druga brez PCA.

Tabela 28: Globalni PCA komponente (Large, brez sortiranja)

Model	PCA komponente (ohranjenih)
PCA + LS	3
Samo LS	/

Pri globalnem modelu s PCA smo obdržali 3 glavne komponente, ki pojasnijo 95.0% variance. Po implementaciji PCA smo s pomočjo metode LS naredili predikcijski model, kjer smo prišli do naslednjih rezultatov:

Tabela 29: RMSE za globalni model PCA + LS (učenje in testiranje) - Large (brez sortiranja)

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.068297	0.088657
1	0.012771	0.007479
2	0.059909	0.093427
3	0.029913	0.143339

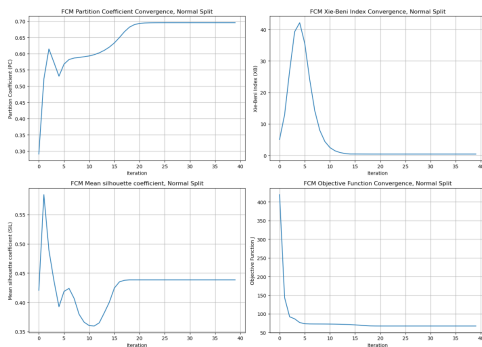
Tabela 30: RMSE za globalni model samo LS (učenje in testiranje) - Large (brez sortiranja)

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.066579	0.066697
1	0.010817	0.008306
2	0.029111	0.060853
3	0.026082	0.025778

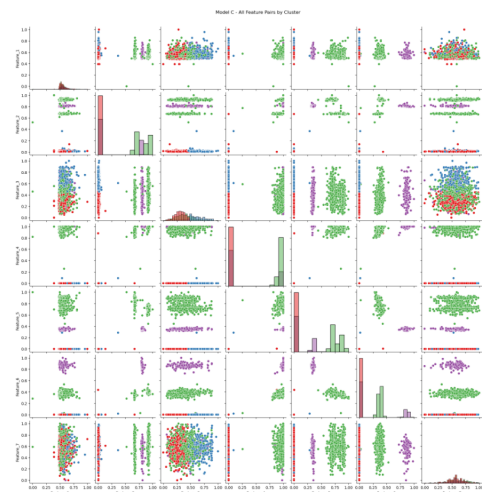
Iz tabele 29 in 30 je razvidno, da model brez PCA dosega boljše rezultate. Pri učenju RMSE doseže 0.1006 (samo LS) v primerjavi z 0.1077 (PCA + LS). Pri testni množici pa dosežemo 0.1176 (samo LS), pri PCA + LS pa dosežemo RMSE vrednost 0.1351. Prav tako so $RMSE_{mean}$ vrednosti po razredih nižje pri modelu brez PCA, še posebej pri razredu 3 na testni množici, kjer model brez PCA doseže 0.025778, medtem ko PCA + LS doseže 0.143339. Vendar je treba upoštevati, da ima razred 3 le 6 testnih primerov.

3.4.3 Model C

Pri Modelu C smo ponovili postopek iz Modela A, vendar tokrat z uporabo vseh značilk, ne samo 2 in 6. Tu smo prišli do naslednjih rezultatov:



Slika 15: Evalvacija metrik FCM na vseh značilkah (Large, brez sortiranja)



Slika 16: Porazdelitev vzorcev po oznakah (Large, brez sortiranja)

Kot je razvidno iz slik 16 in 15, so evalvacije tokrat slabše, saj SIL ne konvergira proti najvišji vrednosti. Prav tako vrednosti ne konvergirajo proti najvišji/najnižji vrednosti.

Do slabše identifikacije pride tudi zaradi upoštevanja značilk, kot so 1, 3 in 6, ki imajo veliko pokrivanja med razredi, kar lahko prinese negotovost v odločanje.

Kljub temu smo tu poskusili delovanje PCA + LS predikcije in prišli do naslednjih rezultatov:

Tabela 31: PCA komponente po razredih (Model C - Large, brez sortiranja)

Razred	PCA komponente (ohranjenih)
0	4
1	3
2	5
3	3

Po uspešni implementaciji PCA smo s pomočjo metode LS naredili predikcijski model, kjer smo prišli do naslednjih rezultatov:

Tabela 32: RMSE po razredih (učenje in testiranje) - Model C (Large, brez sortiranja)

Razred	$RMSE_{mean}$ (Train)	$RMSE_{mean}$ (Test)
0	0.032429	0.011974
1	0.006102	0.005186
2	0.057881	0.047750
3	0.014890	0.082336

Iz tabele 32 je razvidno, da je $RMSE_{mean}$ pri učenju najnižji za razred 1 (0.006102), medtem ko je $RMSE_{mean}$ pri testiranju prav tako najnižji za razred 1 (0.005186). Najvišjo napako pri testiranju ima razred 3 (0.082336), vendar je treba upoštevati, da ima razred 3 le 7 testnih primerov. Prav tako opazimo, da ima razred 2 največje število ohranjenih PCA komponent (5), kar nakazuje na večjo kompleksnost podatkov v tem razredu.

4 Zaključek

Ugotovili smo, da za pravilno delovanje algoritma AG potrebujemo lepe in čiste podatke ter da je lahko zelo koristen, ko nimamo na razpolago oznak (angl. labels). Prav tako smo videli korist, ki jo predstavlja PCA, saj nam lahko enostavno zmanjša dimenzionalnost. Tekom dela smo tudi prišli do problema izračuna RMSE vrednosti, saj nismo imeli pravilno definirane vektorizacije, kar je povzročilo napake pri izračunih. To smo na koncu popravili.